

PeerStudy Developer Onboarding

Welcome to PeerStudy. This guide is for a student developer opening the repository for the first time. It explains what the folders mean, where the application starts, what `lib/` contains, what `pubspec.yaml` controls, and how the Flutter application communicates with Supabase.

The guide focuses on the files a developer is expected to maintain. Generated platform boilerplate, caches, and build output are explained as groups because Flutter recreates them and they should not be edited file by file.

You do not need to understand every file before making a small change. Start with the project map, follow one feature from its screen to its backend call, and run the checks near the end of this guide.

1. The project in one minute

PeerStudy has two main parts:

1. A Flutter application with runners for Android, iOS, web, Windows, Linux, and macOS. Android is the documented phone-test and CI build target.
2. A Supabase backend for authentication, PostgreSQL data, private file storage, realtime updates, and protected server functions.

The usual flow of a feature is:

```
Student/Admin taps the UI
      |
      v
Screen widget in lib/screens/
      |
      v
Controller/repository in lib/providers/ or service in lib/services/
      |
      v
Supabase Auth, Database, Storage, Realtime, or Edge Function
```

Supabase is the source of truth for accounts and shared academic data. The device stores only a small amount of local information, such as theme choice, the last academic selection, and a safe login marker.

PeerStudy supports exactly two roles:

- **Student:** signs up, browses subjects, reads approved PDFs, takes quizzes, uses subject Communities, and reports content.
- **Admin:** uses a pre-created account to manage academic content, materials, reports, and Student access.

2. Start with these files

When learning the repository, read these files in this order:

1. `README.md` - setup commands and the current project summary.
2. `lib/main.dart` - application startup and first-route selection.
3. `lib/routes/app_routes.dart` - the names of all main pages.
4. `lib/routes/app_router.dart` - which widget is created for each route.
5. One file in `lib/screens/` for the feature you want to understand.
6. The provider or service imported by that screen.
7. `docs/architecture.md` - the larger security and data design.

3. Top-level repository map

Path	What it represents	Should a beginner edit it?
lib/	The main Dart/Flutter application source	Yes; most app work happens here
test/	Unit and widget regression tests	Yes; update tests with behavior changes
assets/	Source artwork used by generators or declared as runtime resources	Yes, when intentionally changing an asset
supabase/	Database migrations, seed data, Edge Functions, and backend scripts	Only when changing backend behavior
android/	Android application wrapper, permissions, Gradle, signing, and icons	Only for Android-specific work
ios/	iOS application wrapper, Xcode settings, deep links, and icons	Only for iOS-specific work
web/	Browser entry page, PWA manifest, favicon, and web icons	Only for web-specific work
windows/	Windows desktop runner and CMake configuration	Only for Windows-specific work
linux/	Linux desktop runner and CMake configuration	Only for Linux-specific work
macos/	macOS runner, Xcode settings, entitlements, and icons	Only for macOS-specific work
docs/	Project brief, architecture, setup, audit, and onboarding documentation	Yes
.github/	GitHub Actions automation	Edit when changing CI checks
.vscode/	Shared Visual Studio Code workspace settings	Rarely
.idea/	Local Android Studio/IntelliJ project state	Usually no; most of it is ignored

<code>.dart_tool/</code>	Generated Dart and Flutter tool state	No
<code>build/</code>	Generated test/build output, including APK intermediates	No

The `android/` , `ios/` , `web/` , `windows/` , `linux/` , and `macos/` folders do not contain six separate applications. They are platform wrappers around the same Dart application in `lib/` .

4. What is the `lib/` folder?

`lib` means **library**. In a Flutter application, it contains the Dart source code that is compiled into the app. The normal entry point is `lib/main.dart` .

An import such as:

```
import 'package:peerstudy/services/auth_service.dart';
```

means: start at this project's `lib/` folder, then open `services/auth_service.dart` . The package name `peerstudy` comes from `pubspec.yaml` .

`lib/main.dart`

This is the application entry point. Its startup sequence is:

1. Prepare Flutter plugins.
2. Initialize the public Supabase client.
3. Show a safe configuration screen if initialization fails.
4. Load saved display/settings preferences.
5. Start listening for authentication and password-recovery events.
6. Verify any saved Supabase session and matching profile.
7. Open Login, the Student shell, or the Admin dashboard.

`PeerStudyApp` also configures the light/dark themes, global navigator, and named route generator.

`lib/components/`

Components are reusable UI pieces. They avoid copying the same layout or behavior into many screens.

- `app_button.dart` - the standard primary button, including its loading state.
- `app_form_field.dart` - shared form input styling and password visibility.
- `auth_page_shell.dart` - common responsive layout for authentication pages.
- `editorial_info_page.dart` - reusable layout for informational/legal pages.
- `empty_state_view.dart` - consistent message when a list has no rows.
- `error_view.dart` - consistent error message with an optional retry action.
- `loading_view.dart` - consistent page loading indicator.
- `role_guard.dart` - verifies the current session/profile before protected UI is displayed. The server still performs its own permission checks.

Use an existing component when possible. If the same UI pattern is needed by several screens, it probably belongs here.

lib/config/

- `supabase_config.dart` - public Supabase project URL, public publishable key, and authentication callback URL. These values can be replaced at build time with `--dart-define` .

A Supabase publishable key is a public client identifier. A service-role key, database password, or AI provider key is secret and must never be placed in this folder, Flutter source, an APK, or Git.

lib/models/

Models turn unstructured database/JSON values into named Dart objects.

- `app_user.dart` - verified Student/Admin profile, role, status, and identity.
- `community_attachment.dart` - safe metadata and upload draft values for Community attachments.
- `quiz.dart` - quiz questions, attempts, scores, and correction models.
- `subject.dart` - School, Academic Area, Department, Subject, and material models.

When the backend response changes, update its model validation and tests. Do not pass raw dynamic maps throughout the UI when a model can describe them.

lib/providers/

In this project, these are plain controllers, repositories, or notifiers. The app does not depend on the external Provider, Riverpod, or BLoC packages.

- `academic_provider.dart` - Community posts/comments, reporting, pagination, realtime refresh, and their UI-facing state.
- `quiz_provider.dart` - quiz generation, answer selection, submission, and loading/error state.
- `settings_provider.dart` - theme and last-opened academic path stored locally.
- `subject_provider.dart` - reads the academic hierarchy and remembers the current Area, Department, and Subject selection.

The common pattern is a small state object plus methods that perform an async operation and then notify or return data to a `StatefulWidget` .

lib/routes/

- `app_routes.dart` - constants such as `/login` , `/student-shell` , and `/admin-dashboard` .
- `app_router.dart` - converts each route name into a screen and wraps protected routes with `RoleGuard` .

When adding a new named page, normally add the route constant first, add its case to the router, and then navigate using the constant rather than repeating a string.

lib/screens/auth/

- `login_screen.dart` - Student email/Admin-alias login form.
- `signup_screen.dart` - public Student registration using a LIMU email.
- `forgot_password_screen.dart` - requests a secure password-reset email.
- `reset_password_screen.dart` - accepts a new password from a valid recovery session.

These screens own only form/UI state. `AuthService` owns the actual Supabase authentication rules.

lib/screens/student/

- `student_shell_screen.dart` - bottom-navigation shell for Home, Catalog, and Profile.
- `student_home_screen.dart` - introduction and shortcuts for signed-in Students.
- `student_departments_screen.dart` - Area and Department browsing.

- `student_subjects_screen.dart` - active Subjects for the selected Department.
- `student_subject_workspace_screen.dart` - one Subject workspace containing Materials, Quiz, and Community tabs.
- `material_viewer_screen.dart` - internal viewer for a short-lived private PDF URL.
- `subject_quiz_view.dart` - ten-question quiz UI for one selected material.
- `subject_community_views.dart` - posts, comments, attachments, and private content reporting.

`lib/screens/admin/`

- `admin_dashboard_screen.dart` - Academic and Reports tabs, including content moderation and account restrictions.
- `admin_form_pages.dart` - full-page forms for Areas, Departments, Subjects, and materials.
- `admin_resolution_note_dialog.dart` - lifecycle-safe dialog for the required Admin moderation note.

Admin buttons are not the security boundary. Supabase policies and protected database functions verify the Admin role again.

`lib/screens/profile/`

- `student_profile_screen.dart` - profile summary, recent posts/quiz activity, refresh behavior, and profile links.
- `student_account_screen.dart` - trusted account email, role, and status.
- `settings_screen.dart` - local theme/settings editing.
- `community_guidelines_screen.dart` - Community behavior rules.
- `feedback_screen.dart` - routes content issues to reporting and general help to Support.
- `support_screen.dart` - configured support/contact actions.
- `privacy_policy_screen.dart` - privacy information.
- `terms_screen.dart` - application terms.
- `about_screen.dart` - short PeerStudy product description.

`lib/services/`

Services sit at system boundaries: authentication, database calls, server functions, file storage, and local persistence.

- `supabase_service.dart` - initializes and exposes the single Supabase client.
- `auth_service.dart` - signup, login, logout, recovery, session restoration, and trusted profile loading.
- `backend_api_service.dart` - gateway for protected database RPCs, Storage, Edge Functions, materials, quizzes, attachments, reports, and safe backend errors.
- `login_preference_service.dart` - saves only a harmless login marker and user ID; it does not store passwords or access tokens.
- `settings_storage.dart` - selects the native or browser settings implementation using a conditional export.
- `settings_storage_io.dart` - SQLite settings on Android/iOS, with a SharedPreferences fallback on desktop platforms.
- `settings_storage_web.dart` - browser-compatible settings storage.

Screens should usually call one of these services instead of duplicating raw backend request logic.

`lib/theme/` and `lib/utils/`

- `theme/app_theme.dart` - shared colors, typography, spacing, light/dark themes, and system-bar appearance.
- `utils/validators.dart` - reusable LIMU email and password validation.

Put app-wide visual decisions in the theme and shared input rules in a utility, instead of creating slightly different versions on individual screens.

5. What is `pubspec.yaml` ?

`pubspec.yaml` is the Dart/Flutter project manifest. It tells Flutter the project name, supported Dart version, application version, dependencies, development tools, and declared Flutter resources.

YAML uses indentation to represent structure. Use spaces, keep indentation consistent, and do not use tabs.

Important sections in this project

Section	Meaning in PeerStudy
<code>name</code>	Package/import name: <code>peerstudy</code>
<code>description</code>	Short project description
<code>publish_to: none</code>	Prevents accidental publication as a <code>pub.dev</code> package
<code>version: 1.0.0+1</code>	User-facing version <code>1.0.0</code> , build number <code>1</code>
<code>environment</code>	Requires Dart <code>^3.12.0</code>
<code>dependencies</code>	Packages needed while the app runs
<code>dev_dependencies</code>	Packages used only for tests, linting, or generation
<code>flutter_launcher_icons</code>	Generates consistent icons for all platforms
<code>flutter</code>	Enables Material icons and declares runtime assets/fonts

Runtime dependencies

- `flutter` - Flutter framework itself.
- `cupertino_icons` - iOS-style icon set.
- `supabase_flutter` - Auth, PostgreSQL API, Storage, Realtime, and functions.
- `uuid` - idempotency/request identifiers that make retries safer.
- `file_picker` - selecting PDFs and Community attachments from a device.
- `pdfrx` - internal PDF rendering.
- `url_launcher` - opening approved external/contact or signed-file links.
- `shared_preferences` - small key/value device preferences and login marker.
- `sqlite` - SQLite settings storage on native platforms.
- `crypto` - SHA-256 and related integrity helpers.
- `http` - HTTPS communication helpers.

Development dependencies

- `flutter_test` - Flutter unit and widget test framework.
- `flutter_lints` - recommended Dart/Flutter code-quality rules.
- `flutter_launcher_icons` - generates platform launcher icon files from `assets/branding/peerstudy_app_icon.png`.

After adding or changing a dependency, run:

```
flutter pub get
```

Flutter then updates `pubspec.lock`. For an application like PeerStudy, commit both `pubspec.yaml` and `pubspec.lock` so every developer and CI uses the same resolved package versions. Let Flutter manage the lock file; do not manually write package versions into it.

The current icon source is configured for icon generation. It is not a normal runtime `AssetImage`, because the `flutter.assets` list is currently empty.

6. Other important root files

- `pubspec.lock` - exact dependency versions selected by `flutter pub get`.
- `analysis_options.yaml` - analyzer and lint configuration.
- `.gitignore` - files Git must not track, especially generated output and secrets.
- `.metadata` - Flutter tool metadata; usually leave it unchanged.
- `README.md` - quickest setup and project overview.
- `supabase.example.json` - placeholder public configuration for another Supabase project. Do not run the app with its placeholder values.
- `.github/workflows/flutter_quality.yml` - CI installs dependencies, checks formatting, analyzes, tests, builds an Android APK, and uploads the artifact.

7. What is inside `supabase/` ?

The `supabase/` folder is backend source code. Git stores the definitions, but Git does not contain the live hosted database, user accounts, uploaded files, or deployed secrets.

`supabase/migrations/`

Migrations are timestamped SQL changes applied in order:

- `20260828000100_peerstudy_corrected_master.sql` - core tables, constraints, database functions, role checks, Row Level Security, Storage policies, and Realtime setup.
- `20260830000100_community_attachments.sql` - private Community attachment lifecycle and policies.
- `20260902000100_admin_report_delete_safety.sql` - safe, idempotent Admin resolution when reported content is already removed or missing.

Once a migration has been applied to a shared database, create a new migration for the next change. Do not silently rewrite old migration history.

`supabase/functions/`

These TypeScript Edge Functions perform protected server-side work:

- `generate-quiz/` - generates a real ten-question quiz from approved material.
- `submit-quiz/` - scores a completed quiz and returns corrections.
- `finalize-community-attachment/` - verifies attachment ownership, metadata, bytes, signature, size, and checksum.
- `cleanup-community-attachments/` - removes private bytes that are no longer readable.
- `_shared/` - validation and HTTP helpers shared by several functions.

`supabase/functions/.env.example` documents required server variables. The real `.env` is ignored and must never be committed.

Remaining Supabase files

- `config.toml` - local Supabase CLI services, ports, Auth settings, and Edge Function definitions.

- `seed.sql` - academic reference data only. It deliberately creates no users, passwords, posts, materials, quizzes, or reports.
- `scripts/bootstrap-admin.ts` - trusted operator helper for provisioning the test Admin without placing a service-role key in Git.
- `scripts/hosted-smoke.ts` - hosted acceptance checks and cleanup for major backend flows.

Before applying a database change, preview it:

```
npx.cmd --yes supabase@2.116.0 db push --linked --dry-run
```

Applying a migration changes the shared hosted backend. Confirm the project and obtain the project owner's approval before running the command without `--dry-run`.

8. What is inside `test/` ?

Tests describe behavior that must keep working. A fixed bug should normally get a focused regression test here.

- `auth_role_guard_test.dart` - validation, profile parsing, roles, and protected-route access.
- `subject_hierarchy_test.dart` - academic hierarchy and Subject model parsing.
- `quiz_model_test.dart` and `quiz_provider_test.dart` - quiz contracts and controller flow.
- `academic_provider_serialization_test.dart` - Community and report model serialization.
- `community_attachment_test.dart` - attachment limits, types, and metadata.
- `backend_function_error_test.dart` - safe Edge Function error conversion.
- `login_preference_service_test.dart` - saved-login marker behavior.
- `settings_provider_test.dart` - local settings persistence and rollback.
- `student_profile_account_test.dart` - Profile/Account activity loading, refresh, retry, and navigation.
- `admin_report_moderation_test.dart` - Admin note dialog and safe reported content deletion behavior.
- `widget_test.dart` - application startup and role-guard smoke coverage.

Run the full suite with `flutter test`, or pass one test path while working on a specific feature.

9. Supporting project folders

`assets/`

The current source asset is `assets/branding/peerstudy_app_icon.png`. The launcher-icon generator reads it and writes platform-specific copies. If you later display an image with `Image.asset`, also declare its path under `flutter.assets` in `pubspec.yaml`.

`docs/`

This folder contains the project brief, architecture, implementation guide, Supabase reference, contribution rules, roadmap, repository audit, and test evidence. Documentation explains the system but is not compiled into the app.

`.github/`

`.github/workflows/flutter_quality.yml` defines the checks GitHub runs on pushes and pull requests: dependency install, format check, analysis, tests, release APK build, and artifact upload. A local change passing on one laptop can still fail CI if generated files, formatting, or platform build inputs differ.

10. Platform folders

Most daily Flutter work does not require touching a platform folder.

- `android/` - Android package ID, internet/deep-link permissions, Gradle, optional signing configuration, launcher resources, and Kotlin entry point.
- `ios/` - Xcode project, app information, URL scheme, launch screen, icons, and Swift application entry points.
- `web/` - HTML bootstrap page, installable web manifest, and browser icons.
- `windows/` - C++ runner, application resources, and CMake files.
- `linux/` - Linux runner and CMake files.
- `macos/` - Xcode project, Swift runner, sandbox entitlements, and icons.

Examples of valid platform-specific changes include adding a permission, registering an authentication callback scheme, configuring signing, or regenerating icons. Avoid editing generated plugin registrant files; Flutter recreates them from `pubspec.yaml`.

11. Generated and local-only files

These files may exist on one developer's computer and not another:

- `.dart_tool/` - dependency/tool metadata.
- `.flutter-plugins-dependencies` - generated plugin information.
- `build/` - compiled intermediates, test cache, APKs, and other output.
- `*.log` - local diagnostic logs.
- `root *.apk` files - local phone-test handoff builds.
- `android/key.properties`, `*.jks`, and `*.keystore` - private signing data.
- `supabase/.temp/` - local Supabase project-link/cache state.
- `supabase/functions/.env` - local server secrets.

Do not solve a missing generated file by committing another developer's build folder, IDE cache, signing key, or `.env`. Regenerate dependencies/build output with Flutter instead.

12. Follow a feature through the code

Login example

```
login_screen.dart
-> AuthService.signIn(...)
-> Supabase Auth
-> profiles table validation
-> AppRouter opens Student shell or Admin dashboard
```

Student quiz example

```
subject_quiz_view.dart
-> QuizController in quiz_provider.dart
-> BackendApiService
-> generate-quiz / submit-quiz Edge Function
-> protected database rows
```

Community report example

```
subject_community_views.dart
-> Academic/Community controller or BackendApiService
-> create_content_report database function
-> reports table
-> admin_dashboard_screen.dart moderation action
-> admin_resolve_report database function
```

Profile refresh example

```
student_profile_screen.dart
-> StudentActivityReader
-> community_posts and quiz_attempts
-> loading, available, empty, or retryable error UI
```

13. Where should a change go?

Desired change	Start here
Change one page's layout	Its file under lib/screens/
Reuse a button/form/error pattern	lib/components/
Change global colors or text styles	lib/theme/app_theme.dart
Add or parse a domain object	lib/models/
Change loading/state behavior	Relevant file in lib/providers/ or screen state
Add a protected backend operation	lib/services/backend_api_service.dart plus a migration/function
Change signup/login/session behavior	lib/services/auth_service.dart
Change input rules	lib/utils/validators.dart and matching backend constraint

Add a route	lib/routes/app_routes.dart and app_router.dart
Change a table, policy, trigger, or RPC	New file in supabase/migrations/
Change quiz/attachment server logic	Matching folder in supabase/functions/
Prevent a bug from returning	Add a focused file/test under test/

14. Small Dart and Flutter vocabulary

- `Widget` - one piece of UI.
- `StatelessWidget` - UI that has no changing local state.
- `StatefulWidget` - UI paired with a `State` object that can call `setState`.
- `Future<T>` - a value that will arrive later, often after a database/network operation.
- `async` / `await` - readable syntax for waiting for a `Future`.
- `Stream<T>` - values that can arrive repeatedly, such as Auth or Realtime events.
- `const` - an object Flutter can reuse because its values do not change.
- `required` - the caller must supply this named argument.
- `Type?` - the value may be `null`; code must handle the missing case.
- `_name` - a private Dart class, method, or variable visible only in its file.
- `mounted` - whether a widget's `State` is still on screen; check it after awaited work before calling `setState` or navigating.
- RLS - Row Level Security; database rules deciding which rows a session may read or change.
- RPC - a protected PostgreSQL function called through the Supabase API.
- Edge Function - server-side TypeScript code deployed separately from Flutter.

15. Normal developer workflow

From the repository root:

```
flutter doctor -v
flutter pub get
flutter run
```

Before handing off a change:

```
dart format lib test
flutter analyze
flutter test
git diff --check
git status --short
```

Useful focused commands include:

```
flutter test test\student_profile_account_test.dart
flutter test test\admin_report_moderation_test.dart
flutter build apk
```

Read every `git status` and `git diff` before committing. Generated output and secrets must not appear in the commit.

16. Safety rules for this project

1. Never place a service-role key, AI key, database password, signing key, or user password in Flutter code or Git.
2. A public button/route guard is helpful UX, but permissions must also be enforced by RLS or a protected server function.
3. Treat backend values as nullable/untrusted until validated.
4. Catch network/API failures and show a loading, empty, or retryable error state instead of allowing Flutter's red error screen.
5. Prevent duplicate taps while an async write is running.
6. After `await`, check `mounted` before using a widget's context/state.
7. Use idempotency keys for retryable create/submit operations.
8. Keep answer keys and privileged operations on the server.
9. Create a new migration for an already-deployed database change.
10. Add a regression test whenever you fix a bug.

17. Suggested first exercise

To become comfortable without changing backend behavior:

1. Run the tests.
2. Open `lib/screens/profile/about_screen.dart`.
3. Change one sentence locally.
4. Run `dart format lib test` and `flutter analyze`.
5. Inspect `git diff` to see exactly what changed.
6. Revert or commit only after reviewing the diff with the project owner.

Then trace one real feature using the examples in section 12.

18. Related documentation

- [README](#) - quick start and build commands.
- [Project brief](#) - product scope and roles.
- [Architecture](#) - runtime components and security boundaries.
- [Implementation guide](#) - complete environment/backend setup.
- [Supabase backend reference](#) - database and Edge Function reference.
- [Repository audit](#) - verification evidence and remaining checks.
- [Roadmap](#) - delivery phases and acceptance checklist.
- [Contributing](#) - collaboration and contribution expectations.